8 Puzzle Solver

Informed and Uninformed Search

Table of Contents

1. Algorithms

I. BFS

```
def bfs(game: Game):
    initial_state = State(game.start)
    frontier = deque([initial_state])
    frontier_dict = {initial_state: True}
    nodes_expanded = 0
    max_depth = 0
    found = False
    while frontier:
        state = frontier.popleft()
        frontier_dict[state] = False
        max_depth = max(max_depth, state.depth)
        game.current = state.position
        nodes_expanded += 1
        if state.position == game.end:
            found = True
            return frontier, nodes_expanded, max_depth, found, state
        action_list = ['up', 'down', 'left', 'right']
        for action in action_list:
            new_state = State(game.move(action))
            if new_state.position != game.current:
                new_state.move = action
                new_state.parent = state
                new_state.depth = state.depth + 1
                state.children.append(new_state)
        for child in state.children:
            if child not in frontier_dict:
                frontier_dict[child] = True
                frontier.append(child)
    return frontier, nodes_expanded, max_depth, found, 0
```

Breadth First Search is an uninformed search algorithm that finds the shallowest goal by searching level by level. It is an optimal and complete search that unfortunately is exponential in both time and space complexity.

The frontier is a Queue (First In First Out) and starts initially with the root state.

frontier_dict is a dictionary that keeps track of that states entering and leaving the frontier. A new entry is added every time a node enters the frontier, and it is marked as true. When a state is popped from the frontier, that state is marked as false in the dictionary. This means that the dictionary contains all states that were either explored or still in frontier and therefore is used in the add children check.

At the start of the search, a state is popped following FIFO. It is then marked as such in the dictionary, and the number of nodes expanded increases by 1.

This state then enters the goal check. If it is not the goal it enters the next stage of adding its children states to the frontier.

The 8 puzzle has a maximum of 4 moves possible. Each move is tested against the current game state. Invalid moves are discarded, and valid moves will create new states whose depth, parent state, and action creating the state will be recorded. These new states are then added to the popped state as children.

A new state will only be added to the frontier if they follow these two conditions:

- The new state does not exist in the frontier
- The new state was never explored

These 2 conditions are true if the new state does not exist in the dictionary and therefore that is the condition used.

After the check, the new state(s) will be added to both the frontier and dictionary.

Throughout the search, the maximum depth is kept track of by finding the max() of each incoming state. This information is not useful in BFS since the depth will always be equal to number of actions taken to reach goal. However, keeping track of depth is very important in the coming search algorithms.

Search loop terminates early if the goal state is found. Otherwise, it stops in failure after the frontier is emptied with no goal found.

II. DFS

```
def dfs(game: Game, limit = -1):
    initial_state = State(game.start)
    frontier = deque([initial_state])
    frontier_dict = {initial_state: initial_state.depth}
    nodes_expanded = 0
    max_depth = 0
    found = False
    while frontier:
        state = frontier.pop()
        max_depth = max(max_depth, state.depth)
        game.current = state.position
        nodes_expanded += 1
        if state.position == game.end:
            found = True
            return frontier, nodes_expanded, max_depth, found, state
        if state.depth < limit or limit == -1:
            action_list = ['up', 'down', 'left', 'right']
            for action in action_list:
                new_state = State(game.move(action))
                if new_state.position != game.current:
                    new_state.move = action
                    new_state.parent = state
                    new_state.depth = state.depth + 1
                    state.children.append(new_state)
            for child in state.children:
                if child not in frontier_dict or (child.depth <
frontier_dict.get(child, 0) and limit != -1):
                    frontier_dict[child] = child.depth
                    frontier.append(child)
    return frontier, nodes_expanded, max_depth, found, 0
```

Depth First Search is an uninformed search algorithm that tries to find the goal by moving along the depth of the search tree. It is not optimal, and also has an exponential time complexity, but its favorable linear space complexity makes it very useful when in the Iterative Deepening DFS.

The algorithm follows BFS to the letter except for one major difference: The frontier will be a Stack (Last In First Out). This time the frontier will pop the last element that enters each time to make depth the priority during search.

The above code contains modifications to allow for Depth Limited Search, which is DFS but with a maximum limit of depth put making it only search a part of the search space.

The main modification is that an if condition is used that does not allow the add children part of the code to run if the current state popped from the frontier has a depth equal to the limit.

Another important modification is that for DLS, the old condition to add children (new state cannot be added unless it is not in frontier or visited lists) is counterproductive and can make the search miss goals that can be found within the depth limit. This is

because for the 8 puzzle, it is very probable that a game state can repeat at further and further depths. This means that a state, whose expansions can lead to the goal, maybe first explored while close to the depth limit, not giving it enough expansions to get the goal on the first try. A condition is required to make sure that a state which has been explored before can re-enter the frontier while also not making the search susceptible to infinite loops / waste of search time.

This secondary condition is to check, while adding children of a state, that the child state has smaller depth than that in the frontier or visited lists.

The idea behind making sure that the depth of the repeating state is smaller than the one in visited list is to make sure that the next time this specific state is explored, it will have more expansions than the previous times which may lead to the goal hidden within the given depth limit. This means that a repeating state will enter frontier and branch out again as long as it will yield a extra game states searched and no redundant searches will occur.

The code does also check if the new state will repeat in the frontier. However, since it is impossible to expand a state of low depth unless all states of higher depth are explored first, it is impossible that a duplicate state will appear in frontier due to secondary condition and therefore using our dictionary is fine.

This time the dictionary will track the depth of the nodes which will help in fulfilling the secondary condition.

This function is written in such a way that if no limit is given as an argument, it will ignore the new modifications added for DLS.

III. Iterative Deepening DFS

```
def iddfs(game: Game, iter = float('inf')):
    i = 0
    nodes_expanded = 0
    while i < iter:
        frontier, explored, max_depth, found , state = dfs(game, i)
        nodes_expanded += explored
        i += 1
        if found:
            return frontier, nodes_expanded, max_depth, found , state
    return 0, nodes_expanded, iter - 1, False, 0
```

Iterative Deepening DFS (IDDFS) is an uninformed search algorithm that combines the optimality and completeness of BFS with the low space complexity of the DFS. It works by iterating over the depth limit of the previously mentioned DLS until the depth limit reaches the goal depth where it will be guaranteed to find the goal at its shallowest depth.

New search trees are created each iteration, and DFS is run on them up to the iterating limit. The search trees must be fully explored before going into the next iteration. When the goal is finally reached the loop will terminate.

Maximum iterations functionality is given to only allow the search to iterate up to a certain limit set by the user. The only fail condition is if the iteration limit is lower than the shallowest goal depth.

IV. A*

```
def a_star(game: Game, heuristic):
    initial_state = State(game.start, 'A*')
    initial_state.h = heuristic(game.start, game.end)
    frontier = []
    entry_count = 1
    heapq.heappush(frontier, (initial_state, entry_count))
    frontier_dict = {initial_state: initial_state.cost()}
    visited = set()
    max_depth = 0
    found = False
    while frontier:
        state = heapq.heappop(frontier)[0]
        if state in visited:
            continue
        max_depth = max(max_depth, state.depth)
        game.current = state.position
        visited.add(state)
        if state.position == game.end:
            found = True
            return frontier, len(visited), max_depth, found, state
        action_list = ['up', 'down', 'left', 'right']
        for action in action_list:
            new_state = State(game.move(action), 'A*')
            if new_state.position != game.current:
                new_state.move = action
                new_state.parent = state
                new_state.depth = state.depth + 1
                new_state.g = state.g + 1
                new_state.h = heuristic(new_state.position, game.end)
                state.children.append(new_state)
        for child in state.children:
            if child not in visited:
                if child not in frontier_dict or child.cost() <
frontier_dict.get(child, 0):
                    entry_count += 1
                    heapq.heappush(frontier, (child, entry_count))
                    frontier_dict[child] = child.cost()
    return frontier, len(visited), max_depth, found, 0
```

A* is an informed search algorithm that uses an approximate heuristic (distance from current state to goal state) to help find the goal. The cost function of the A* is the sum of the cost to reach current state from root (g(n)) and the heuristic (h(n)). Its performance and optimality depend on the heuristic used. A* is only optimal if the heuristic underestimates the actual cost for all possible states in the state space. A heuristic that follows such condition is called an admissible heuristic. Given a number of admissible heuristics, the heuristic that provides the best performance will be the one that provides the largest heuristic over all states (and therefore is the closest to the actual cost).

The frontier of the A* must be a priority queue that takes the cost function of a state as its priority. The code above also uses the time of entry as a tie break if two states of equal priority exist in the frontier.

A list of explored nodes by itself is required this time for a certain check and therefore was added in the set ‘visited’.

The dictionary this time contains the cost function of each state and will be used to facilitate the functions of a priority queue.

When adding children, new attributes must be given for the A* search, namely g(n) and h(n). g(n) of a state will always be equal to g(parent) + 1, since each action in the 8 puzzle will only cost 1 unit.

When adding children, the condition to enter frontier will once again change to accommodate the priority added the queue. Two conditions must be followed:

1. The new state must never have been visited before
2. The new state must not be already in the frontier or, the new state must have a lower cost function than its copy in the frontier

The check for visited and frontier cannot be combined this time due to extra condition that can allow a new state to enter frontier at lower cost.

A glaring issue here is that the frontier will contain duplicate states of differing costs, meaning that useless searching will occur the times the state is visited after the first (which is the least cost one). This is solved by checking if the state popped from the frontier has already been visited before executing rest of the code. Any time a state that has been visited before pops out of frontier at higher cost, the code will ignore that state and continue into the next loop.

Of the two given heuristics, the Manhattan heuristic yields much better performance when compared to the Euclidean heuristic. This is simply because Manhattan distance is always greater than Euclidean distance (moving in x and y will always take longer than moving in straight line from one point to another).

2. Data Structures

The 2 built-in hash table data type, set and dictionary, are used due to their O(1) time complexity lookups, which is the most used operation in the search, mainly when checking the explored and frontier before adding new states.

The queue and stack used by BFS and DFS respectively are made using the deque (Double Ended Queue) data structure which, unlike lists, allow popping the first or the last element at the same time complexity.

The priority queue used by A* is made using heapq data structure (Heap Queue). It uses a binary heap that satisfies the min-heap property (lowest value node is at start of queue). It allows pushing and popping from the heap at O(log n) complexity while maintaining the heap structure. The heap structure is perfect for a priority queue with minimum priority popping.

3. Performance

The following performance measures will be compared between each of the discussed search algorithms:

- Path to goal
- Cost of path (will be equal to length of path i.e number of actions)
- Nodes expanded (number of states in visited list)
- Search depth
- Running time

An initial state vector maps to 3x3 matrix:

$$[1,2,3,4,5,6,7,8,0] = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{bmatrix}$$

Initial state: [6, 4, 3, 8, 1, 2, 5, 7, 0]

Optimal path: up -> up -> left -> left -> down -> right -> up -> left -> down -> right -> right -> down -> left -> left -> up -> right -> up -> right -> down -> left -> down -> right -> up -> left -> up -> left

	Search Algorithm				
Performance Measure	BFS	DFS	IDDFS	A* Manhattan	A* Euclidean
Path (Y/N)	Y	N	Y	Y	Y
Cost	26	10668	26	26	26
Nodes Expanded	129381	172562	1128855	2909	105343
Search Depth	26	61728	26	26	26
Running Time (s)	2.788	3.255	14.534	0.0928	5.353

Initial state: [7, 5, 3, 8, 1, 6, 2, 4, 0]

Optimal path: left -> left -> up -> right -> right -> up -> left -> left -> down -> right -> right -> down -> left -> left -> up -> right -> up -> right -> down -> down -> left -> up -> up -> right -> down -> left -> up -> left

	Search Algorithm				
Performance Measure	BFS	DFS	IDDFS	A* Manhattan	A* Euclidean
Path (Y/N)	Y	N	Y	Y	Y
Cost	28	46064	28	28	28
Nodes Expanded	171924	54804	1562900	4499	141314
Search Depth	28	46066	28	28	28
Running Time (s)	3.389	1.377	20.118	0.0928	6.762

Initial state: [2, 1, 0, 3, 5, 4, 6, 7, 8]

Optimal path: left -> down -> right -> up -> left -> left -> down -> right -> right -> up -> left -> down -> left -> up

	Search Algorithm				
Performance Measure	BFS	DFS	IDDFS	A* Manhattan	A* Euclidean
Path (Y/N)	Y	N	Y	Y	Y
Cost	14	21122	14	14	14
Nodes Expanded	2935	162889	7500	105	1941
Search Depth	14	61541	14	14	14
Running Time (s)	0.0311	2.499	0.056145	0.00351	0.079594

4. References

https://www.andrew.cmu.edu/course/15-121/lectures/Binary%20Heaps/heaps.html

https://www.geeksforgeeks.org/heap-queue-or-heapq-in-python/

https://docs.python.org/3/library/heapq.html

https://stackabuse.com/stacks-and-queues-in-python/

https://www.geeksforgeeks.org/stack-and-queues-in-python/

https://www.geeksforgeeks.org/sets-in-python/

https://www.freecodecamp.org/news/python-set-vs-list/